INTERVIEW PREPARATION GUIDE

Sanket Ajay Chopade — DevOps Engineer

CI/CD · AWS · Kubernetes · IaC · Observability · DevSecOps

34 curated questions across 6 domains

Follow-Up · Cross-Domain · Scenario-Based · Troubleshooting

Contents

Question Types

Follow-Up	Cross	Scenario-Based	Troubleshooting
Builds on stated experience	Connects multiple skill areas	Situational judgment & design	Problem-solving under pressure

1. CI/CD & Pipeline Engineering

FOLLOW-UP

Q1.1

You reduced deployment time from 45 to 10 minutes. Walk through every stage you added, removed, or parallelised to achieve that.

■ MODEL ANSWER

The original pipeline ran all stages sequentially: checkout → build → test → Docker build → push → deploy. I parallelised the Maven unit tests across modules using Jenkins parallel stages, which cut test time by ~40%. I replaced the full Docker build (rebuilding from scratch each run) with layer-cached multi-stage builds, slashing image build time from ~12 min to under 3 min. I also eliminated a manual approval gate between staging and production that was adding ~8 min of idle wait. The Kubernetes rollout used kubectl apply with --wait=false initially, so I changed to a rollout status check with a timeout, catching failures faster instead of the pipeline hanging. Combined, those changes took the cycle from 45 to 10 minutes.

CROSS

Q1.2

Your Jenkins pipelines push images to ECR, and Terraform provisions your EKS cluster. How do you ensure a Terraform-managed EKS change doesn't break a running Jenkins deployment mid-flight?

■ MODEL ANSWER

This is a real race-condition risk. My approach: Terraform changes to EKS node groups or control-plane config are gated behind a separate pipeline with a manual approval step and a maintenance-window tag. Jenkins pipelines check a DynamoDB-backed state flag ('maintenance_in_progress') at pipeline start and abort gracefully if set. For rolling node-group updates, I use Terraform's create_before_destroy lifecycle with PodDisruptionBudgets on critical workloads, so Kubernetes drains nodes safely before Terraform destroys old ones. Blue-green cluster upgrades go further — I spin up the new node group, drain the old one, then destroy it, all coordinated through Terraform workspaces and a pipeline mutex.

SCENARIO-BASED

Q1.3

A Jenkins pipeline passes in staging but fails silently in production — the deploy step exits 0 but the new pods are CrashLoopBackOff. How do you redesign the pipeline to catch this?

■ MODEL ANSWER

Silent success is worse than an explicit failure. I'd add a post-deploy verification stage: after kubectl rollout apply, I run kubectl rollout status deployment/ --timeout=180s, which blocks and returns non-zero if pods don't reach Ready within the timeout. I also add a smoke-test stage — a curl against the service's /health endpoint through the ALB — before marking the build green. In the Kubernetes layer, I'd enforce readinessProbe and livenessProbe on every Deployment so Kubernetes itself won't route traffic to a crashing pod. On the Jenkins side, I'd add a post-failure step that runs kubectl describe pod and kubectl logs --previous and publishes the output as a build artifact, so the root cause is visible without needing cluster access.

TROUBLESHOOTING

Q1.4

Your GitHub Actions workflow starts failing with 'rate limit exceeded' on docker pull during the build step. How do you diagnose and fix this without changing the workflow logic?

■ MODEL ANSWER

Docker Hub unauthenticated pulls are limited to 100/6h per IP. Diagnosis: check workflow logs for the exact error — 'toomanyrequests' from registry-1.docker.io confirms it. Fix options in priority order: (1) Authenticate — add DOCKERHUB_USERNAME and DOCKERHUB_TOKEN as GitHub secrets and add a docker/login-action step before any pull; this raises the limit to 200/6h per account. (2) Mirror to ECR — use ECR Public or a private ECR repo as a pull-through cache; update base image references in Dockerfiles to point to the ECR mirror. (3) Cache layers — use actions/cache to cache Docker layers by Dockerfile hash, so repeated builds don't re-pull. Option 2 is the most robust for a team environment as it eliminates the dependency on Docker Hub entirely.

2. AWS Infrastructure & Terraform

FOLLOW-UP

Q2.1

You used 'modular Terraform' for AWS provisioning. Describe the module structure — what's in a root module vs a child module, and how do you handle cross-module dependencies like passing an EKS cluster ARN to an RDS security group?

■ MODEL ANSWER

Root module (environments/prod/main.tf) only calls child modules and wires outputs to inputs — no direct resource blocks. Child modules (modules/eks, modules/rds, modules/vpc, modules/alb) each own their resource definitions and expose outputs. Cross-module dependencies: the vpc module outputs private_subnet_ids and vpc_id; the eks module takes those as inputs; the rds module takes the eks module's worker_security_group_id as an input to its security group ingress rule. This explicit wiring means Terraform's dependency graph is correct — rds won't plan before eks outputs exist. I avoid data sources across modules where possible because they create implicit dependencies that aren't reflected in the graph, making plan output misleading. State is split per environment in S3 with DynamoDB locking; remote state references (terraform_remote_state) are used sparingly and only for genuinely shared infrastructure like VPC.

SCENARIO-BASED

Q2.2

You need to migrate a production RDS instance to a larger instance class with zero downtime. Terraform says it needs to destroy and recreate the instance. How do you handle this?

■ MODEL ANSWER

Terraform's default behaviour for certain RDS changes is replace, which is destructive. My approach: first, check whether the change actually requires replacement — instance_class changes in RDS are applied in-place if apply_immediately=false (they happen during the next maintenance window). If Terraform is incorrectly planning a replace, I use lifecycle { prevent_destroy = true } as a safeguard and check the plan output carefully. For the actual change, I set apply_immediately=false and let it apply during the maintenance window to avoid a forced restart. If a same-day change is needed, I use an RDS blue-green deployment (available for MySQL/PostgreSQL): provision the new larger instance, replicate from the current one, test against the replica endpoint, then do a DNS-level cutover by updating the CNAME in Route53 — zero downtime, and rollback is trivially reversing the DNS change.

CROSS

Q2.3

You manage IAM via Terraform. How do you reconcile Terraform-managed IAM with the need for developers to temporarily escalate permissions without drifting state?

■ MODEL ANSWER

Permanent permissions stay in Terraform — any manually created IAM policy shows up as drift in terraform plan and gets flagged in the CI pipeline. For temporary escalation, I use AWS IAM Identity Center (SSO) permission sets with time-limited assignments, or AWS STS AssumeRole with a max session duration — neither of these is tracked in Terraform state, so there's no drift. We have a break-glass role in Terraform with prevent_destroy and a CloudTrail alarm on any assume-role of it, so emergency access is available but audited. The key principle: Terraform owns the structure (roles, policies, trust relationships); runtime access decisions (who can assume a role right now) live in Identity Center or STS, outside Terraform's concern.

TROUBLESHOOTING

Q2.4

terraform apply exits with 'Error: Provider configuration not present' in a CI pipeline that worked yesterday. What's your diagnosis process?

■ MODEL ANSWER

This error means Terraform can't find a provider block for a resource it's trying to manage — usually caused by a state/config mismatch. Diagnosis steps: (1) Run terraform providers to see what providers the configuration expects vs what's in the lock file. (2) Check if someone removed a provider block or moved a resource to a new module without a corresponding required_providers entry. (3) Check if the remote state was modified externally — another pipeline may have applied a change that added a resource whose provider isn't in this configuration. (4) Check the .terraform.lock.hcl for version pin changes. Most common fix: run terraform init -upgrade in the CI environment to re-download providers and regenerate the lock file. If the issue is a moved resource, use a moved block in the config to tell Terraform about the relocation without destroying/recreating the resource.

3. Kubernetes & Container Orchestration

FOLLOW-UP

Q3.1

You implemented blue-green deployments on EKS. Walk me through the exact Kubernetes objects involved and how traffic shifts from blue to green.

■ MODEL ANSWER

Blue-green on Kubernetes without a service mesh: I run two Deployments — app-blue and app-green — each with a distinct label (version: blue / version: green). A single Service uses a selector that matches one label at a time. To shift traffic: update the Service's selector from version: blue to version: green — this is an atomic, zero-downtime switch since Kubernetes immediately updates the endpoint slice. Before cutting over, I validate the green deployment is fully Ready (kubectl rollout status), run smoke tests against the green pods directly via port-forward, and only then update the Service. Rollback is re-patching the selector back to blue — takes under 5 seconds. I automate this in Jenkins with kubectl patch service. For more granular traffic splitting, I'd use an Ingress controller annotation (NGINX weighted canary) or move to AWS App Mesh.

CROSS

Q3.2

Your resume mentions HPA alongside resource limits. How do you tune HPA thresholds so the autoscaler doesn't fight against resource limits and cause OOMKills instead of scaling out?

■ MODEL ANSWER

The HPA and resource limits operate at different layers and can conflict if misconfigured. The most common failure: memory requests/limits set too low, so pods hit the memory limit and get OOMKilled before HPA has time to scale out (HPA has a sync period of 15s by default). My approach: set requests conservatively based on observed P95 memory usage from Prometheus metrics, set limits at ~150% of requests to allow bursting. HPA for CPU targets 60-70% of the request (not limit) — this gives headroom for bursting while triggering scale-out before saturation. For memory, I avoid memory-based HPA because memory is not compressible — an OOMKill is instant, and HPA can't react fast enough. Instead, I use Vertical Pod Autoscaler (VPA) in recommendation mode to inform right-sizing and keep memory headroom adequate. I also set --horizontal-pod-autoscaler-downscale-stabilization to 5 minutes to prevent flapping.

SCENARIO-BASED

Q3.3

A microservice pod in EKS is intermittently evicted during peak load. On investigation, you see the node is hitting memory pressure. How do you systematically fix this without just increasing node size?

■ MODEL ANSWER

Scaling up the node is the expensive shortcut that hides the root cause. Systematic approach: (1) Identify the evicted pod's resource profile — kubectl describe pod shows the eviction reason (memory). Check if the pod has no resource requests set (Kubernetes treats these as BestEffort QoS — first to be evicted). (2) Set resource requests so the pod gets Burstable or Guaranteed QoS. (3) Check for memory leaks — pull heap metrics from the application's /actuator/metrics (Spring Boot) or custom Prometheus metrics. If memory grows monotonically without GC, it's a leak, not a sizing issue. (4) Use LimitRange on the namespace to enforce minimum requests for all pods. (5) If the node legitimately runs out, check if HPA is scaling pods before the cluster autoscaler adds nodes — if so, tune the cluster autoscaler's scale-up threshold or pre-warm node groups. (6) Add node-level memory pressure alerting in Grafana so you catch this before evictions happen.

TROUBLESHOOTING

Q3.4

kubectl get pods shows a pod stuck in 'Pending' state indefinitely on EKS. Walk me through your complete diagnosis.

■ MODEL ANSWER

kubectl describe pod is the first command — the Events section almost always contains the specific reason. Common causes and their signatures: (1) Insufficient resources: 'Insufficient cpu' or 'Insufficient memory' — the scheduler can't place the pod on any node. Check node capacity with kubectl describe nodes and whether the cluster autoscaler is enabled (check its logs in kube-system). (2) Taints/tolerations mismatch: 'No nodes available to schedule pods' despite available capacity — the pod lacks a toleration for a node taint. (3) NodeSelector/affinity mismatch: the pod requires a label that no node has. (4) PVC pending: if the pod has a PVC, check kubectl get pvc — if the PVC is also Pending, the storage class provisioner may be failing (check its pods). (5) Namespace resource quota exceeded: kubectl describe namespace shows the quota. Fix depends on root cause — I work through these top to bottom based on the Events output.

4. Monitoring, Observability & DevSecOps

FOLLOW-UP

Q4.1

You claim zero critical vulnerabilities across all production deployments using Trivy and SonarQube. What's your enforcement mechanism — how do you ensure a developer can't bypass the scan?

■ MODEL ANSWER

Policy enforcement needs to live in the pipeline, not in human review. In Jenkins, the Trivy scan stage runs before the Docker push step and uses trivy image --exit-code 1 --severity CRITICAL — a non-zero exit code fails the pipeline and blocks the push to ECR. SonarQube Quality Gate is enforced via the SonarQube Jenkins plugin's waitForQualityGate() call, which polls the Quality Gate status and fails the build if it doesn't pass. To prevent bypass: branch protection rules on GitHub require the Jenkins CI status check to pass before merge to main, and only the CI service account has push permissions on the production ECR repository — developers cannot push images directly. The only bypass would be a Jenkins admin disabling the stage, which is audited. For SAST, SonarQube's Quality Gate is version-controlled as a profile, so developers can't weaken it without a code review.

SCENARIO-BASED

Q4.2

MTTR is 30% lower thanks to your observability stack. Describe a specific incident where Prometheus/Grafana alerts fired before users noticed an issue — and walk through how you triaged it.

■ MODEL ANSWER

A concrete example: a Spring Boot service's JVM heap was growing 8% per hour due to a connection pool leak. The Grafana alert (JVM heap > 85% for 10 minutes) fired at 2 AM before any user-visible latency impact. Triage: I checked the Grafana dashboard — heap was growing without GC reclaiming it, classic leak pattern. Checked Prometheus metrics for DB connection pool (HikariCP metrics exposed via Actuator): active connections were pinned at max (20/20) with no idle connections. Cross-referenced Datadog APM traces: all requests to a specific endpoint were hanging at the DB layer. Root cause: a code change had removed the try-with-resources block from a DAO method, leaking connections. Immediate mitigation: rolling restart of the pods (cleared the heap), feature flag to disable the leaking endpoint. Permanent fix: code change + SonarQube rule added to detect unclosed resources. Total MTTR: 22 minutes from alert to mitigation.

CROSS

Q4.3

You run ELK stack alongside Prometheus/Grafana and Datadog. That's three overlapping observability tools. How do you justify the cost and avoid alert fatigue from duplicate alerts across systems?

■ MODEL ANSWER

Fair challenge — this is a common trap. The justified split: Prometheus/Grafana owns infrastructure and application metrics (CPU, memory, request rate, error rate) and is the source of truth for SLO dashboards. ELK owns log aggregation and search — structured application logs with trace IDs for correlation. Datadog was evaluated for APM distributed tracing during a PoC because Prometheus doesn't natively support distributed traces. To avoid duplication: alerting rules live exclusively in Alertmanager (Prometheus) for infrastructure alerts and PagerDuty for on-call routing — ELK and Datadog are query/exploration tools, not alert sources. This means one alert fires per condition. Realistic cost justification: Datadog was a trial that would be replaced by OpenTelemetry + Jaeger once the PoC concluded. For a production decision, I'd consolidate on Prometheus + Loki + Tempo (the Grafana stack) to reduce cost and eliminate the overlap.

TROUBLESHOOTING

Q4.4

Grafana dashboards show a sudden spike in HTTP 502 errors from your ALB to EKS services. Prometheus shows pod metrics are healthy. Where's the disconnect and how do you isolate it?

■ MODEL ANSWER

ALB 502 means the ALB received a bad response from the target — the target is alive but returning an error. Since pod metrics look healthy, the issue is likely at the networking layer between ALB and pods, not inside the pods. Isolation steps: (1) Check ALB access logs in S3 — look at the target_status_code field. If it's '-', the connection was refused or timed out before the pod responded. (2) Check ALB target group health in the AWS console — are any targets marked unhealthy? (3) If targets are healthy but 502s persist, check if the pod's containerPort matches the Service's targetPort and the target group's port. (4) Check Security Group rules — if a recent Terraform apply modified the worker node SG, it may have removed the ALB's inbound rule. (5) Check readinessProbe — if the probe is passing but the application isn't actually ready (e.g., still initialising a cache), it could be accepting connections it can't serve. Run kubectl logs on the affected pods during the spike to correlate.

5. Architecture & System Design

SCENARIO-BASED

Q5.1

You're asked to design a disaster recovery plan for the ApnaKart platform with an RTO of 1 hour and RPO of 15 minutes. What's your architecture?

■ MODEL ANSWER

RPO of 15 minutes drives the data strategy; RTO of 1 hour drives automation. Data layer: RDS Multi-AZ is baseline (synchronous replication, automatic failover, ~seconds RPO within a region). For cross-region RPO of 15 min, enable RDS automated backups to S3 and set up RDS cross-region read replica in a secondary region; promote it on failover. S3 data uses cross-region replication with versioning. Infrastructure layer: Terraform configuration is stored in Git — the entire AWS footprint can be reproduced from code. I'd maintain a warm standby EKS cluster in the secondary region (not active, but nodes provisioned) to avoid cold-start time. DNS: Route53 health checks on the primary ALB; failover routing record points to secondary ALB. On failure, Route53 auto-promotes the secondary in ~30-60 seconds. RTO breakdown: DNS failover (~1 min) + RDS promotion (~5-10 min) + app deployment on warm cluster (~10-15 min) = well within 1 hour. Runbooks are tested quarterly with a chaos game day.

CROSS

Q5.2

Your resume shows both GitHub Actions and Jenkins. For a greenfield project today, which would you choose and why — and when would you choose the other?

■ MODEL ANSWER

For a greenfield cloud-native project: GitHub Actions, without hesitation. Reasons: zero infrastructure to maintain (no Jenkins server to patch, backup, or scale), native integration with the source repository, marketplace of pre-built actions, and built-in secrets management. The cost model (usage-based minutes) is cheaper than running EC2 instances 24/7 for Jenkins workers at small-to-medium scale. When I'd choose Jenkins: (1) Air-gapped or on-premise environments where GitHub-hosted runners are not an option. (2) Very complex pipeline orchestration with plugins that have no GitHub Actions equivalent (e.g., heavy Groovy-based pipeline logic with shared libraries already in use). (3) Organisations with existing Jenkins expertise and large shared libraries — migration cost outweighs the benefit. (4) Pipelines that need to orchestrate across multiple VCS systems. The honest answer is that Jenkins is the right choice when you're maintaining an existing investment, not when starting fresh.

SCENARIO-BASED

Q5.3

A product team wants to move from a monolith to microservices and asks you to design the CI/CD strategy. What are the first three things you establish before writing a single pipeline?

■ MODEL ANSWER

Three foundations before any pipeline code: (1) Deployment unit definition — each microservice needs a clear ownership boundary: its own repository (or a clearly partitioned mono-repo), its own Dockerfile, its own versioned container image. Without this, 'microservice CI/CD' is just a monolith with more pipelines. (2) Environment promotion strategy — define the environment chain (dev → staging → production) and the promotion gates (automated tests pass, manual approval, canary threshold). This determines pipeline structure more than any tooling choice. (3) Shared infrastructure vs per-service infrastructure — decide what's shared (EKS cluster, VPC, observability stack) vs per-service (Kubernetes namespace, IAM service account, RDS schema). The boundary determines how Terraform modules and pipeline permissions are structured. Getting these wrong means rearchitecting pipelines after they're in production, which is expensive.

TROUBLESHOOTING

Q5.4

After a Kubernetes cluster upgrade from 1.27 to 1.28 on EKS, several Helm-managed deployments start failing. How do you systematically identify which charts are incompatible?

■ MODEL ANSWER

Kubernetes version upgrades deprecate and remove APIs — this is the most common Helm compatibility break. Systematic approach: (1) Before the upgrade, run Pluto or kubent (kube no-trouble) against the cluster to scan all deployed manifests for deprecated API versions. This gives you a pre-upgrade hit list. (2) After the upgrade, for failing deployments: helm list -A to see all releases, then helm status to see which are in a failed state. (3) For each failed release: kubectl describe deployment shows the API version in the manifest; cross-reference with the Kubernetes deprecation guide. Common example: Ingress moved from networking.k8s.io/v1beta1 to networking.k8s.io/v1 in 1.22 and was removed — older ingress-nginx charts break. (4) Fix: upgrade the Helm chart version (usually the chart maintainer has published a compatible version). If not: pull the chart, patch the apiVersion in the template, and push to an internal chart museum. (5) Add a Pluto scan to the CI pipeline so this is caught before the next upgrade.

6. Behavioural & Situational

SCENARIO-BASED

Q6.1

A developer pushes directly to main, bypassing your CI pipeline, and the deployment breaks production. How do you handle the immediate situation and prevent recurrence?

■ MODEL ANSWER

Immediate: revert the commit with git revert, trigger the pipeline to redeploy the last known-good artifact — don't try to fix forward under pressure. Communicate the incident status to stakeholders immediately with an ETA. Prevention: this is a process failure, not a people failure. Enable branch protection on main: require status checks (CI pipeline) to pass and at least one code review before merge. Restrict direct push permission on main to a service account only. Add a CODEOWNERS file so changes to critical paths require specific reviewers. Post-incident: run a blameless retrospective focused on the system gap, not the individual. Document it as an incident report. The conversation with the developer is private and focused on the workflow, not punitive — the goal is to make the safe path also the easy path.

SCENARIO-BASED

Q6.2

You're halfway through a Terraform migration when you discover the existing infrastructure has manual changes that aren't in state. You have two hours before a deployment freeze. What do you do?

■ MODEL ANSWER

Two hours is not enough to safely reconcile unknown drift before a freeze — deploying into uncertainty is how you cause an outage. My call: halt the migration, communicate to the team that the freeze timeline is at risk, and explain why. Then triage the drift: run terraform plan and catalogue every resource Terraform wants to change. For each unmanaged resource, decide: import it (terraform import) if it's simple and well-understood, or mark it as excluded with a lifecycle ignore block temporarily to unblock the deployment. Document every deviation. After the freeze lifts, run a dedicated drift-reconciliation session with the infrastructure owner who made the manual changes. The lesson for the future: run terraform plan as a read-only drift check in CI on a schedule, so drift is caught continuously, not discovered at migration time.